

Tuning a Random Forest: GridSearchCV and RandomizedSearchCV

Contents

1. Overview	2
2. The heart disease data	3
3. A random forest out of the box	3
4. One split is not enough	4
5. Tuning one setting by hand	5
6. Grid search over a random forest	6
6.1 The grid	6
6.2 Running the search	6
6.3 The result	7
6.4 Did tuning really help?	7
7. Randomized search	8
7.1 A bigger grid	8
7.2 A trap: bootstrap and max_samples	8
7.3 The result	9
8. Summary	10
9. Sources	10
10. Key terms	11

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate), 10 (Tune). See the [Course map](#).



- **Builds on:** Simple imputation (mean, median, mode, constant) ([Note 37](#)); ML pipelines ([Note 38](#)); Overfitting ([Note 91](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)).
- **Leads to:** Stacking and blending ([Note 127](#)); Random under- and oversampling ([Note 133](#)); Balanced random forest ([Note 133](#)); Optuna ([Note 134](#)).
- **Compare with:** Train-test split ([Note 13](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)); Optuna ([Note 134](#)); Bayesian optimisation ([Note 134](#)); Dropout ([Note 1024](#)).

1. Overview

Key point: A random forest is strong out of the box. A grid search or a randomized search, scored by cross-validation, tunes its hyperparameters systematically; on a small dataset like ours, the tuned forest does no better than the default one.

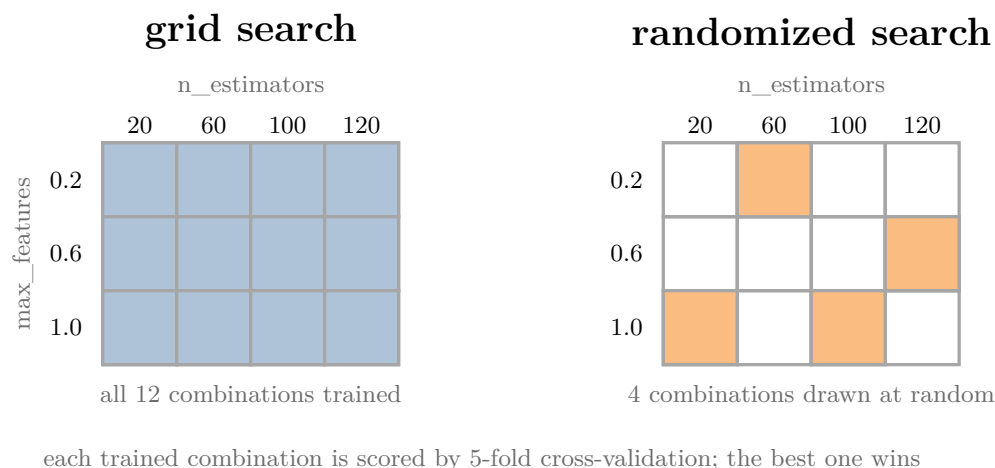


Figure 1: Grid search trains every combination of the listed values; randomized search trains a few combinations drawn at random

This Note applies a random forest to a real dataset, heart disease, and compares it with other algorithms. Then the Note tunes the forest's hyperparameters (the [random forest hyperparameters Note](#)) with the two search methods of Figure 1.

Both searches were introduced earlier: `GridSearchCV` in the [pipelines Note](#), section 9, and `RandomizedSearchCV` in the [regression trees Note](#), section 7.2. Here we see what a random forest grid looks like, how long it takes, and one trap specific to forests.

The Notebook (`notebook.ipynb`) runs every step.

2. The heart disease data

Key point: 303 patients, 13 medical features, and a target: 1 for heart disease, 0 for none.

The data is a well-known heart disease dataset from Kaggle. Each of the 303 **observations** (records, one row of the data table each) is a patient. Its **features** (input variables, one column each) are age, sex, chest pain type (`cp`), resting blood pressure (`trestbps`), cholesterol (`chol`), maximum heart rate (`thalach`) and other medical measurements. The last column, **target**, is the **target** (the output we predict): whether the patient has heart disease.

Predicting a class in this way is a classification problem: from the 13 features, predict the target. We hold out 20% for testing: 242 training observations and 61 test observations.

3. A random forest out of the box

Key point: With no tuning at all, the random forest scores 0.836 on the test split and 0.832 with cross-validation, the best cross-validated score of the four algorithms.

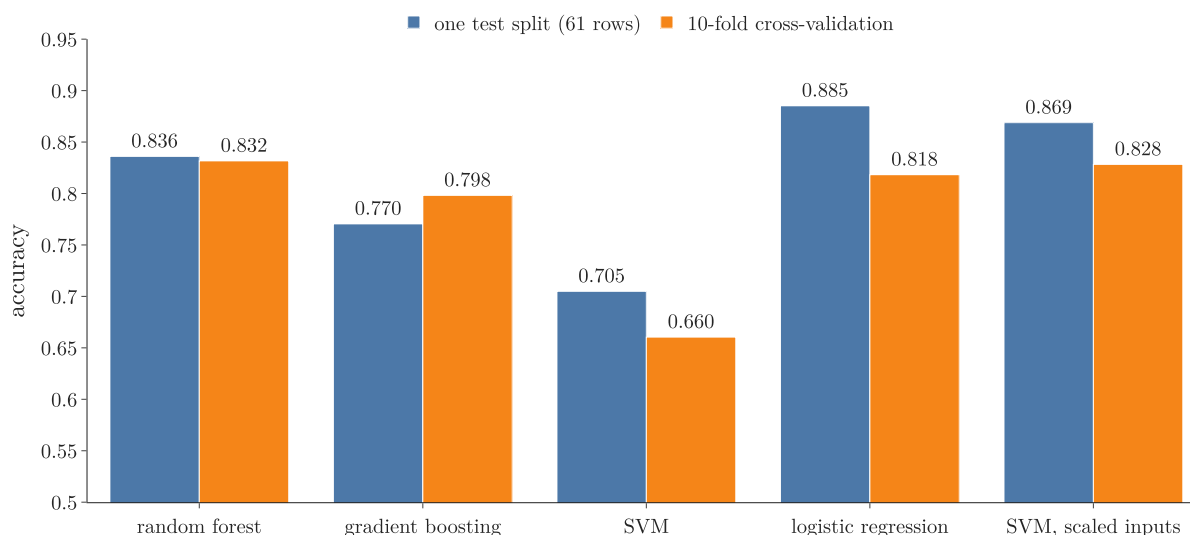


Figure 2: Accuracy of five classifiers, all with default settings, on one test split and with 10-fold cross-validation

We train four classifiers with their default settings and score each on the 61 test observations (Figure 2, blue):

Model	Test split	10-fold cross-validation
Random forest	0.836	0.832
Gradient boosting	0.770	0.798
SVM (SVC)	0.705	0.660
Logistic regression	0.885	0.818

Gradient boosting is another tree ensemble, covered in later Notes; it is used here in the same way as any scikit-learn model. On the single test split, logistic regression wins; section 4 shows why that split is not the whole story.

The random forest does not always win. But a large study of 179 classifiers on 121 datasets found random forests the family most likely to come out on top (Fernández-Delgado et al. 2014), so an untuned forest is a strong first model.

Extra: The SVM's poor score is not a fair verdict on SVMs. An SVM measures distances, so it needs scaled inputs (the [KNN Note](#), section 3.2, shows the same for KNN), and cholesterol, in the hundreds, swamps the 0/1 features. With a `StandardScaler` in a pipeline, the SVM scores 0.869 on the split and 0.828 with cross-validation (Figure 2, right). A random forest needs no scaling: a tree compares one feature at a time with a threshold, so rescaling a feature moves the threshold but not which observations fall on each side. (ESL §10.7, Table 10.1). In the Notebook, putting a `StandardScaler` in front of the forest leaves its scores almost unchanged (0.836 and 0.835). Logistic regression on unscaled data needs `max_iter=5000` to converge; with the default 100 iterations, scikit-learn warns that it stopped early.

4. One split is not enough

Key point: 61 test observations give a noisy score; 10-fold cross-validation trains and tests 10 times and averages, a more reliable number.

A single split of 61 observations can be lucky or unlucky. Cross-validation (the [pipelines Note](#), section 8) averages the scores of 10 train-and-test rounds, so it gives a steadier number.

With cross-validation (Figure 2, orange), logistic regression drops from 0.885 to **0.818**, while the random forest stays at **0.832**. The two are in fact close, and the forest is slightly ahead.

Python: 10-fold cross-validation.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(RandomForestClassifier(random_state=42),
                          X, y, cv=10)
scores.mean()      # 0.832
cross_val_score returns the 10 accuracies; their mean is the cross-validated score.
```

5. Tuning one setting by hand

Key point: Giving each tree fewer observations makes the trees less alike, and the forest a little more accurate: 0.826 with full-size samples, 0.834 with 20% of the observations per tree.

`max_samples` sets how many training observations each tree gets (the [random forest hyperparameters Note](#), section 3). The default, `None`, gives each tree a bootstrap sample as large as the training set.

The idea. When every tree sees a large sample, the samples overlap a lot, so the trees learn much the same thing and make the same mistakes. Averaging copies of the same mistake does not cancel it. Smaller samples overlap less, so the trees differ more, and their mistakes cancel more often in the vote. The [bagging vs random forest Note](#) gives the formula: the less alike the trees, the lower the forest’s variance (ESL §15.2). Breiman’s bound on a forest’s error says the same: error falls when the trees are less correlated, as long as each tree stays reasonably accurate (Breiman 2001, Thm 2.3).

The result. We change only `max_samples` and score each forest with 10-fold cross-validation, averaged over 20 runs:

Observations per tree (<code>max_samples</code>)					
	all (<code>None</code>)	75%	50%	30%	20%
Mean cross-validated accuracy	0.826	0.825	0.829	0.833	0.834

Smaller samples score higher, and on the same folds 20% of the observations beats all of them in 16 of the 20 runs. The trees really do become less alike: the average correlation between two trees’ predictions drops from 0.44 to 0.34.

Extra: The gain is small, about 2 or 3 of 303 patients, while a single cross-validation run moves by about 0.01 when only the seed changes. Because of this noise, the Notebook averages 20 runs, each with new folds and a new forest, and uses 500 trees so the forest’s own randomness stays small. Too few observations hurt again, because each tree becomes too weak: the [random forest hyperparameters Note](#) shows a forest with 25 observations per tree losing accuracy. The best share depends on the data: on that Note’s demo data the score is flat from about a quarter of the observations, on the heart data about 20% is best.

So a setting can matter, and the problem is scale: a random forest has about 20 hyperparameters, and guessing a good value for each by hand is hopeless. We need a systematic search.

6. Grid search over a random forest

Key point: Four hyperparameters with 4, 3, 3 and 3 values make 108 forests; 5-fold cross-validation trains each 5 times, 540 fits in all, 13 to 22 seconds here.

6.1 The grid

Key point: List the values to try for each hyperparameter; the search trains a forest for every combination.

We choose values for the four settings that matter most:

Python: The grid for a random forest.

```
param_grid = {
    "n_estimators": [20, 60, 100, 120],  # trees
    "max_features": [0.2, 0.6, 1.0],      # columns per split
    "max_depth": [2, 8, None],            # depth of each tree
    "max_samples": [0.5, 0.75, 1.0],     # rows per tree
}
```

The grid is a **dictionary**: each key is a hyperparameter's name, exactly as the model spells it, and each value is the list of values to try.

The search tries every combination, so the number of forests is the product of the list lengths:

$$4 \times 3 \times 3 \times 3 = 108 \text{ forests}$$

With 4 hyperparameters, the combinations form a 4-dimensional table (Figure 1 shows 2 of the dimensions); every cell becomes one forest.

6.2 Running the search

Key point: GridSearchCV takes the model, the grid, the number of folds, and optionally verbose and n_jobs.

Python: Grid search with 5-fold cross-validation.

```
from sklearn.model_selection import GridSearchCV

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
                        param_grid, cv=5,
                        verbose=1, n_jobs=-1)
rf_grid.fit(X_train, y_train)
# Fitting 5 folds for each of 108 candidates,
# totalling 540 fits
cv=5: every forest is trained and scored 5 times. verbose=1 prints the progress line;
n_jobs=-1 uses every CPU core, which matters when hundreds of forests are trained.
```

The search trains $108 \times 5 = 540$ forests. On this small dataset that takes 13 to 22 seconds on 12 cores, depending on how busy the machine is; on a large dataset it can take hours.

6.3 The result

Key point: The best forest has 20 trees of depth 8, 20% of the features per split and 75% of the observations per tree: cross-validated accuracy 0.843.

`rf_grid.best_params_` gives the winning combination, and `rf_grid.best_score_` its mean cross-validated accuracy:

<code>n_estimators</code>	<code>max_features</code>	<code>max_depth</code>	<code>max_samples</code>	Cross-validated accuracy
20	0.2	8	0.75	0.843
20	0.2	None	0.75	0.839
20	0.6	8	1.0	0.831
20	0.2	8	1.0	0.831
60	0.6	8	1.0	0.827

The table lists the top five, from `rf_grid.cv_results_` (the [missing indicator Note](#)). The winner scores **0.869** on the 61 test observations. By default, `GridSearchCV` retrains the best combination on the whole training set, so `rf_grid` itself predicts with the best forest.

6.4 Did tuning really help?

Key point: Judged fairly, no: the tuned forest scores 0.814 and the default forest 0.819. The grid's own 0.843 is the luckiest of 108 scores.

The grid's 0.843 is the best of 108 scores measured on the same 5 folds, so it is a little lucky, like the tallest of 108 people picked at random: **selection bias** (the [regression trees Note](#), section 7.4). The 0.869 on the test set comes from only 61 patients, one of which moves the score by 0.016.

A fair test is **nested cross-validation**. An outer cross-validation splits the data; inside each outer training part, the whole grid search runs and picks a winner; the winner is then scored on the outer test part, which the search never saw. With 5 outer folds repeated 4 times (20 outer folds):

	Mean accuracy
Default random forest	0.819
Grid-tuned random forest	0.814
The grid's own best score, averaged	0.845

The tuned forest beats the default on only 5 of the 20 folds and ties on 5. The grid's own score, 0.845, overstates the tuned forest by about 0.03.

The result matches a large study: across 38 datasets, the random forest was the least tunable of six common algorithms, its defaults already close to the best settings (Probst et al. 2019). Tuning pays off more for models such as SVMs, the most tunable in that study; the search tools are the same.

7. Randomized search

Key point: RandomizedSearchCV trains only a fixed number of random combinations (10 by default), so a much bigger grid stays affordable.

7.1 A bigger grid

Key point: Three more hyperparameters turn 108 combinations into 864; randomized search tries 10 of them, 50 fits.

We add `bootstrap`, `min_samples_split` and `min_samples_leaf`, with two values each. The bigger grid has $108 \times 2 \times 2 \times 2 = 864$ combinations, or 4,320 fits in a grid search. RandomizedSearchCV tries only `n_iter` of them (the [regression trees Note](#), section 7.2), 10 by default: $10 \times 5 = 50$ fits (Figure 1, right).

7.2 A trap: bootstrap and max_samples

Key point: `bootstrap=False` with a `max_samples` value is not allowed, so half the random combinations fail; the fix is a list of two grids.

Putting both lists into one grid lets the search draw combinations such as `bootstrap=False`, `max_samples=0.5`. scikit-learn refuses these, because `max_samples` only applies when observations are drawn (the [random forest hyperparameters Note](#), section 3.3). The search does not stop: it records the failed combinations with the score NaN (“not a number”) and prints a warning. In our run, **5 of the 10** combinations failed, so the search really tried only 5.

The fix is to pass a **list of grids**: the search picks one of the grids at random each time, then a combination from it.

Python: Two grids, so `max_samples` only meets `bootstrap=True`.

```
from sklearn.model_selection import RandomizedSearchCV

grids = [
    dict(param_grid, bootstrap=[True],
         min_samples_split=[2, 5], min_samples_leaf=[1, 2]),
    {"n_estimators": [20, 60, 100, 120],
     "max_features": [0.2, 0.6, 1.0],
     "max_depth": [2, 8, None], "bootstrap": [False],
     "min_samples_split": [2, 5], "min_samples_leaf": [1, 2]},
]
rf_random = RandomizedSearchCV(
```

```
RandomForestClassifier(random_state=42), grids,  
n_iter=10, cv=5, n_jobs=-1, random_state=42)  
rf_random.fit(X_train, y_train)  
dict(param_grid, bootstrap=[True], ...) copies param_grid and adds keys. The  
second grid has no max_samples, so it keeps the default, None. random_state=42 fixes  
which combinations are drawn.
```

7.3 The result

Key point: In one or two seconds, the randomized search finds a forest with cross-validated accuracy 0.826, a little below the full grid's 0.843.

The best of the 10 combinations: 20 trees, 60% of the features, fully grown, `min_samples_split=5`, `min_samples_leaf=2`, all the observations with replacement. Its cross-validated accuracy is **0.826**, and its test accuracy 0.836.

The randomized search trained 50 forests instead of 540 (and instead of 4,320 for a grid over its larger space). The randomized search found a good forest, with a lower best score than the grid: the speed-for-accuracy trade of the [regression trees Note](#), section 7.2.

8. Summary

	Grid search	Randomized search
Class	<code>GridSearchCV</code>	<code>RandomizedSearchCV</code>
Combinations tried	all (here 108)	<code>n_iter</code> (default 10)
Fits with <code>cv=5</code>	540	50
Our time	13 to 22 seconds	1 to 2 seconds
Best cross-validated accuracy (optimistic)	0.843	0.826
Use when	small data, few hyperparameters	large data, many hyperparameters

- Out of the box, the random forest had the best cross-validated score of four algorithms (0.832).
- Judge models by cross-validation, not one small test split: logistic regression fell from 0.885 to 0.818.
- Fewer observations per tree made the trees less alike and the forest more accurate: 0.826 to 0.834 with 20% of the observations.
- A grid over 4 hyperparameters needed 108 forests and 540 fits; its best score, 0.843, is optimistic.
- Nested cross-validation is the fair test: tuned 0.814 against default 0.819. A random forest's defaults are hard to beat.
- `bootstrap=False` cannot be combined with `max_samples`: use a list of grids.

9. Sources

Built from

- CampusX, “Hyperparameter Tuning Random Forest using GridSearchCV and RandomizedSearchCV | Code Example”, YouTube, <https://www.youtube.com/watch?v=4Im0CT43QxY>

Other references

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Cawley, G. C. and Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research* 11, 2079–2107.
- Fernández-Delgado, M., Cernadas, E., Barro, S. and Amorim, D. (2014). Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research* 15, 3133–3181.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. §10.7 (Table 10.1), §15.2.
- Probst, P., Boulesteix, A.-L. and Bischl, B. (2019). Tunability: importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research* 20(53), 1–32.

10. Key terms

Term	Meaning
Parameter grid	A dictionary of hyperparameter names and the values to try for each
List of grids	Several parameter grids passed together, so incompatible values never meet
Nested cross-validation	Cross-validation with the whole tuning search inside each outer fold, so the chosen model is scored on data the search never saw